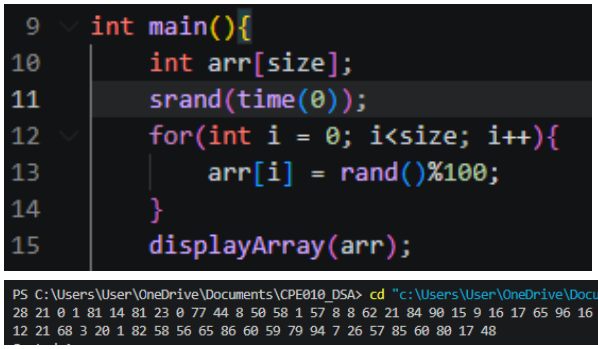
T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES — COMPUTER ENGINEERING — QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - Data Structures and Algorithms**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**09-09-26**	**DATE SUBMITTED:**	**09-16-26**
NAME:	**Christian Lee C. Bandigan**	**INSTRUCTOR:**	**Engr. Jimlord M. Quejado**

ACTIVITY NO. 8 — **Hands-on Activity 8.1 Sorting Algorithms Pt2**

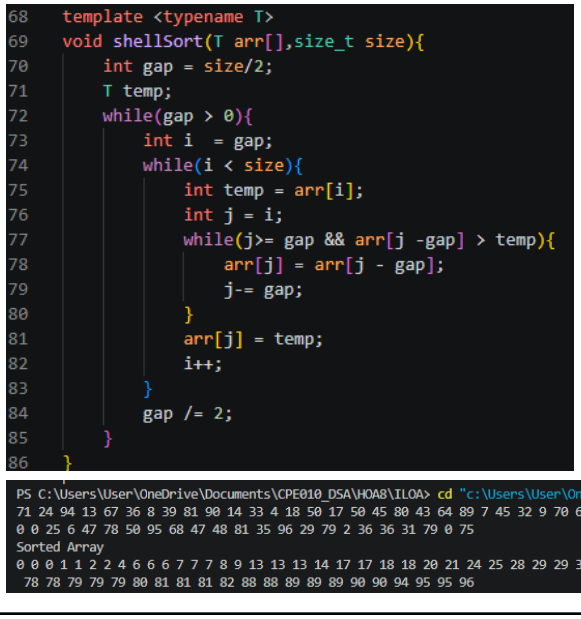
1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	I created an array of unsorted integer values using a for loop and rand with time(0) as the seed.

```
int main(){
    int arr[size];
    srand(time(0));
    for(int i = 0; i<size; i++){
        arr[i] = rand()%100;
    }
    displayArray(arr);
```

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA> cd "c:\Users\User\OneDrive\Documents\CPE010_DSA\HOA8\ILOA\" ; if ($?)
28 21 0 1 81 14 81 23 0 77 44 8 50 58 1 57 8 8 62 21 84 90 15 9 16 17 65 96 16 31 39 11 43 50 30 8 28 96 76 15 83 9 2
12 21 68 3 20 1 82 58 56 65 86 60 59 79 94 7 26 57 85 60 80 17 48
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
template <typename T>
void shellSort(T arr[],size_t size){
    int gap = size/2;
    T temp;
    while(gap > 0){
        int i  = gap;
        while(i < size){
            int temp = arr[i];
            int j = i;
            while(j>= gap && arr[j -gap] > temp){
                arr[j] = arr[j - gap];
                j-= gap;
            }
            arr[j] = temp;
            i++;
        }
        gap /= 2;
    }
}
```

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA8\ILOA> cd "c:\Users\User\OneDrive\Documents\CPE010_DSA\HOA8\I
71 24 94 13 67 36 8 39 81 90 14 33 4 18 50 17 50 45 80 43 64 89 7 45 32 9 70 6 1 42 58 45 41 79 13 17 74 31 53 8
0 0 25 6 47 78 50 95 68 47 48 81 35 96 29 79 2 36 36 31 79 0 75
Sorted Array
0 0 0 1 1 2 2 4 6 6 6 7 7 7 8 9 13 13 13 14 17 17 18 18 20 21 24 25 28 29 29 31 31 32 33 34 35 36 36 36 39 40 41
 78 78 79 79 79 80 81 81 81 82 88 88 89 89 89 90 90 94 95 95 96
```

Observations	Shell sort works by computing for “gap” and comparing and switching values that are “gap” apart. This is basically an improvement to insertion sort because after each iteration the elements move closer to their correct position and when the gap is equal to on it uses an insertion sort on array that is almost sorted. Its time complexity is based on how you compute for gap but normally it is: Best-Case Time Complexity: O(nlogn) Worst-Case Time Complexity: $O(n^2)$

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
100  template <typename T>
101  void merge(T arr[], int left , int mid, int right){
102      int n1 = mid - left +1;
103      int n2 = right - mid;
104      T L[100];
105      T R[100];
106
107      for(int i = 0; i <n1; i++){
108          L[i] = arr[left + i];
109      }
110      for(int j = 0;j<n2;j++){
111          R[j] = arr[mid + 1 + j];
112      }
113
114      int i = 0;
115      int j = 0;
116      int k = left;
117      while(i<n1&& j<n2){
118          if(L[i] <= R[j]){
119              arr[k] = L[i];
120              i++;
121          }else{
122              arr[k] = R[j];
123              j++;
124          }
125          k++;
126      }
127      while(i < n1){
128          arr[k] = L[i];
129          i++;
130          k++;
131      }
132      while(j<n2){
133          arr[k] = R[j];
134          j++;
135          k++;
136      }
137  }
```

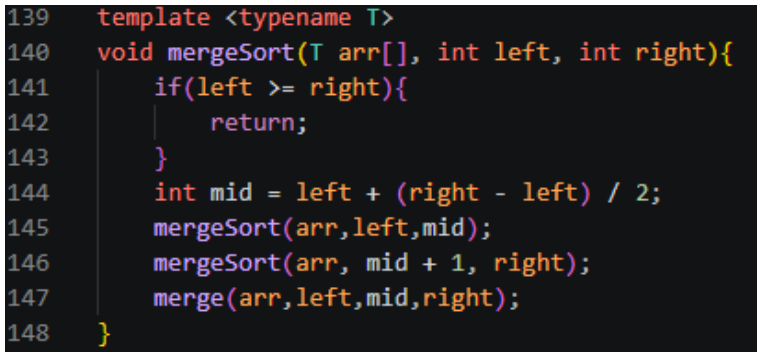
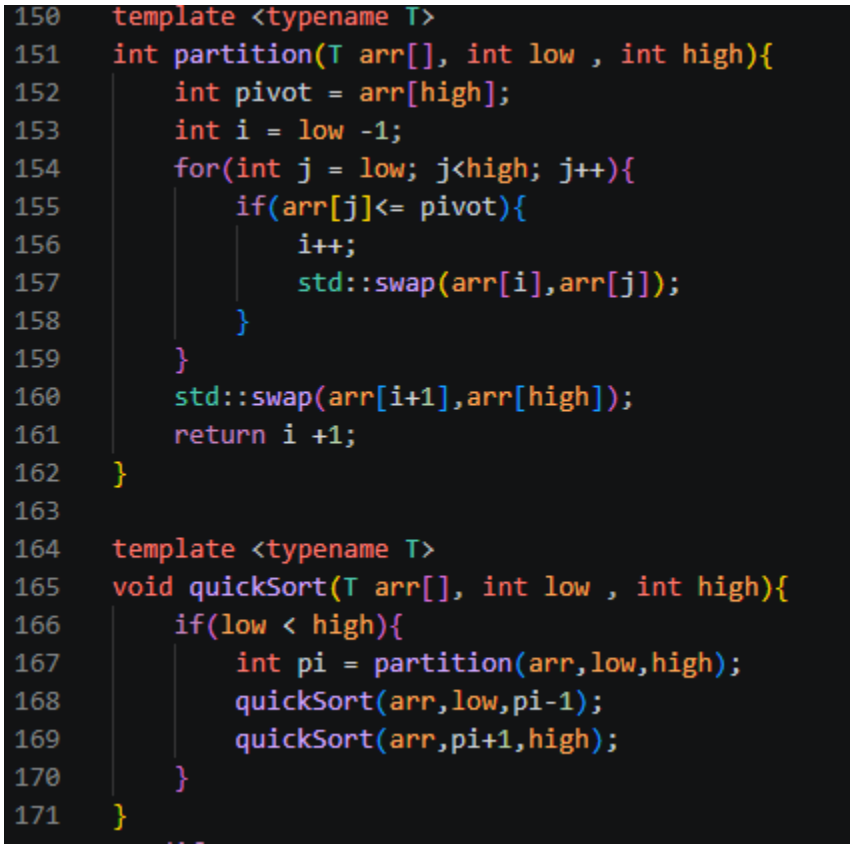
	
Observations	Merge sort works by repeatedly dividing the array into halves and when one value remains it's considered sorted. After that it merges the arrays back until you have a sorted array. Best-Case Time Complexity: O(nlogn) Worst-Case Time Complexity: O(nlogn)

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	
Observations	Quick sort works by selecting a pivot value and after each iteration the pivot value will be in the correct position with elements on its left having smaller values and

	greater values on the right. It repeats for the left and right partitions until you have a sorted array. Best-Case Time Complexity: O(nlogn) Worst-Case Time Complexity: $O(n^2)$

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

We can technically use any sorting algorithms to sort the left and right sub list from the partition method but it can be efficient to use insertion sort especially if it is smaller arrays.

```
5   const int size = 50;
6   void displayArray(int arr[]);
7   int partition(int arr[], int low , int high);
8   void quickSort(int arr[], int low , int high);
9   void insertionSort(int arr[], int high, int low);
10
11  int main(){
12      int arr[size];
13      srand(time(0));
14      for(int i = 0; i<size; i++){
15          arr[i] = rand()%100;
16      }
17      displayArray(arr);
18
19      std::cout << "Modified Quick Sort:" << std::endl;
20      quickSort(arr,0, size-1);
21      displayArray(arr);
22
23      return 0;
24  }
25  void displayArray(int arr[]){
26      for(int i = 0; i<size; i++){
27          std::cout << arr[i] <<" ";
28      }
29      std::cout << std::endl;
30  }
```

```cpp
int partition(int arr[], int low , int high){
    int pivot = arr[high];
    int i = low -1;
    for(int j = low; j<high; j++){
        if(arr[j]<= pivot){
            i++;
            std::swap(arr[i],arr[j]);
        }
    }
    std::swap(arr[i+1],arr[high]);
    return i +1;
}
void insertionSort(int arr[], int high, int low){
    int K = low +1, J;
    int temp;

    while(K<=high){
        temp = arr[K];
        J =K-1;

        while(temp <= arr[J] && J>=low){
            arr[J+1] = arr[J];
            J--;
        }
        arr[J+1] = temp;
        K++;
    }
}
void quickSort(int arr[], int low , int high){
    if(low < high){
        int pi = partition(arr,low,high);
        insertionSort(arr,pi-1,low);
        insertionSort(arr,high,pi+1);
    }
}
```

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA8\ILOA> cd "c:\Users\User\OneDriv
98 99 6 44 92 54 48 24 18 18 42 59 28 1 46 37 66 85 19 47 35 60 67 57 88 94 10 12 2
Modified Quick Sort:
1 3 3 5 6 10 12 16 18 18 19 19 21 21 24 24 26 28 31 35 37 39 42 43 44 46 47 47 48 5
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity

- Based on time complexities, I would say merge sort and quick sort would give the fastest time performance. Merge sort and quick sort have O(nlogn) for their time complexity because they take a constant time to cut the array into halves and they have for/while loops that loop n times. So, O(n) x O(logn) equals O(nlogn).

Conclusion

In this activity I learned more efficient sorting algorithms specifically, shell sort, merge sort, and quick sort. These sorting algorithms are better because they have a better time complexity of O(nlogn). I was also able to implement these sorting algorithms in C++ to sort an array of 100 unsorted elements. Moreover, in the supplementary activity, I was able to answer problems about these sorting algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.